

实验四：微信小程序开发

实验项目基本信息

实验编号：d20301035104、d20301009204
学时分配：4 学时
实验类型：验证型
每组人数：6 人
对应课程目标：课程目标 2

核心步骤列表

实验概览

- 1. 需求分析阶段：绘制小程序页面流程图与数据模型
- 2. AI 辅助编码阶段：使用 TRAE 生成小程序页面代码与数据绑定逻辑
- 3. 测试验证阶段：使用微信开发者工具进行真机预览与功能测试
- 4. 部署运维阶段：体验小程序审核与发布流程（体验版）

开发任务清单

任务阶段	主要内容	关键技术点
通知列表	列表展示	WXML、wx:for
详情页	内容展示	页面路由、参数传递
本地存储	数据持久化	wx.setStorage
AI 辅助	代码生成	TRAE 使用

成功标准

- [] 通知列表页面功能完整
- [] 详情页跳转正常
- [] 数据本地存储功能正常
- [] AI 辅助开发体验良好
- [] 小程序可预览

实验任务与案例

核心任务

借助 AI 编程工具辅助开发通知小程序（列表 + 详情页路由 + 本地存储），体验 AI 工具在代码生成与调试中的作用，完成一个可预览的"智慧校园通知"小程序。

实战案例

开发"智慧校园通知"小程序，展示校园通知列表，点击进入详情页，支持已读状态本地存储，体验小程序完整开发流程。

系统架构设计

本项目采用小程序典型的三层架构设计，视图层（**View**）负责 UI 展示，逻辑层（**Service**）负责业务逻辑处理，数据层（**Model**）负责数据存储与管理，整体架构如下：

系统架构图

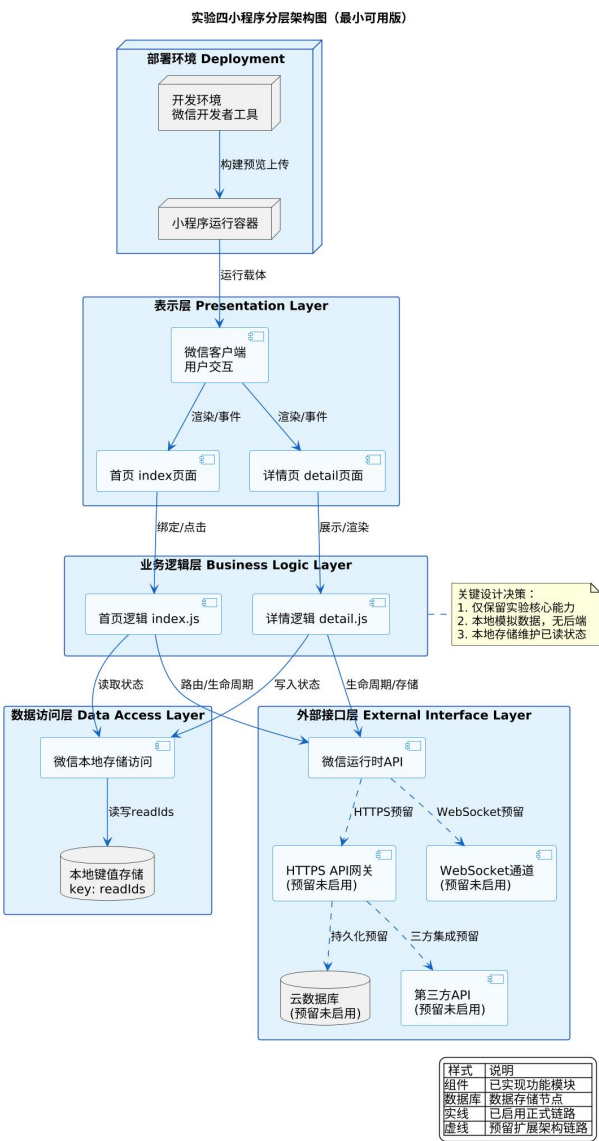


图 1. 微信小程序通知系统整体架构图

核心类结构设计

系统核心页面与数据结构设计遵循小程序开发规范，各页面职责明确，结构如下：

类图

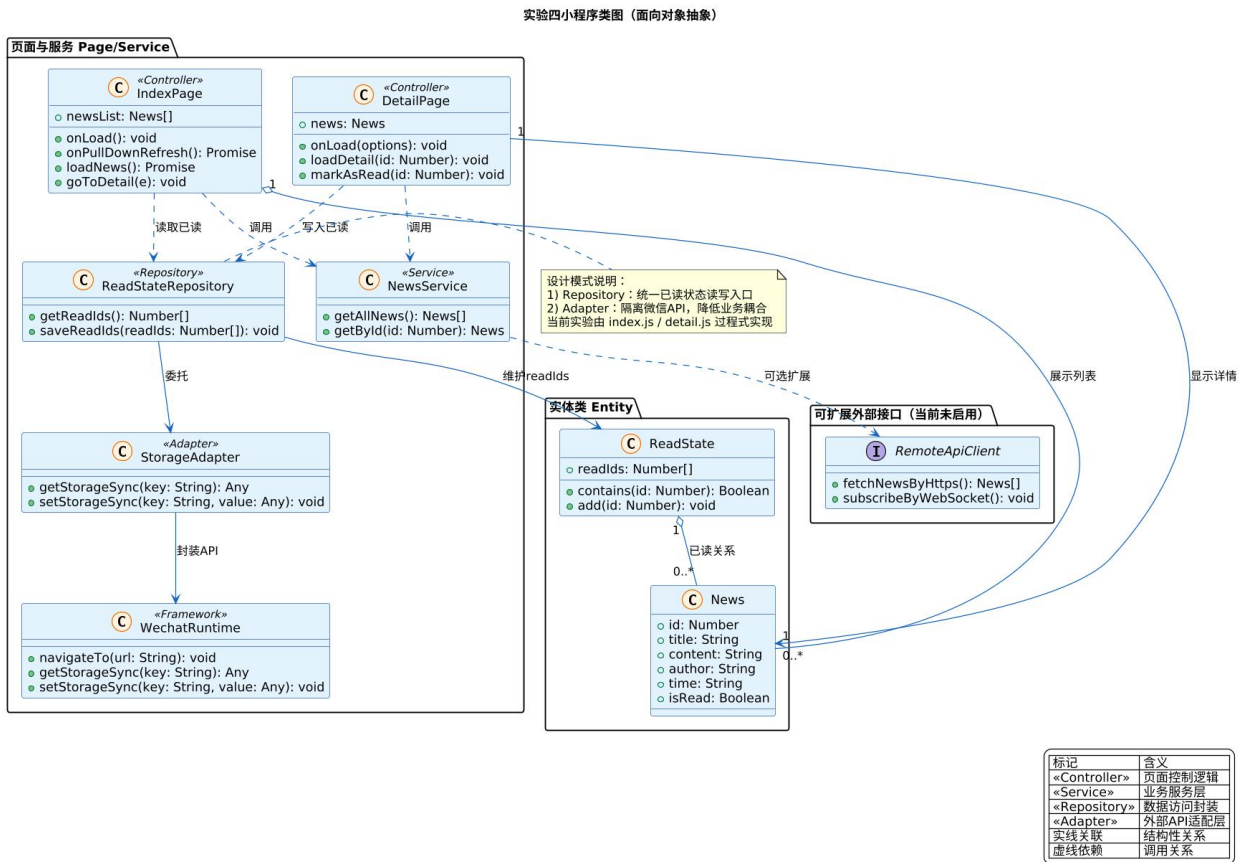


图 2. 微信小程序通知系统核心类结构图

页面流程设计

用户打开小程序后，从首页列表浏览通知，点击进入详情页，系统自动标记已读状态，完整流程如下：

流程图

实验四小程序系统流程图（最小可用版）

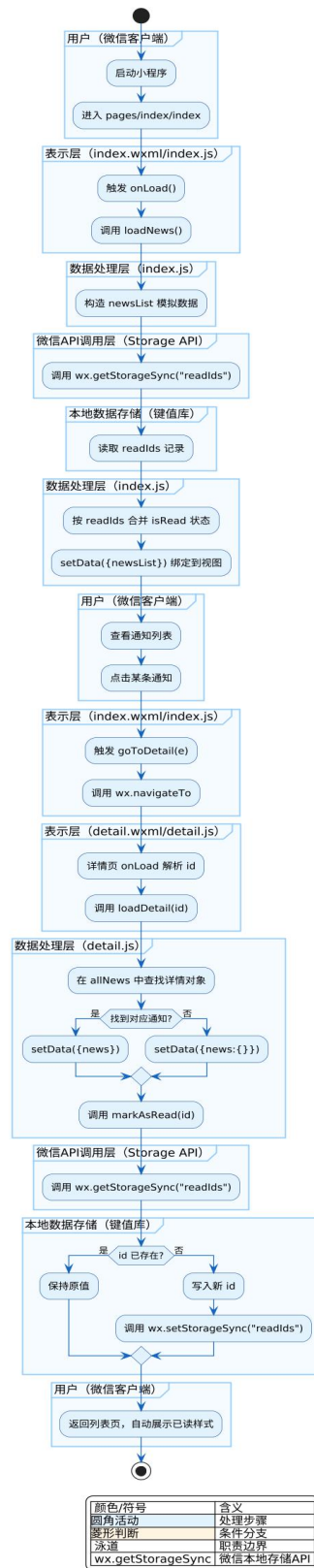


图 3. 微信小程序通知系统页面流程图

第一课时：小程序基础与列表开发（2 学时）

4.1 需求分析阶段

步骤 1：定义数据模型

// 通知数据结构

```
{
  id: 1,
  title: "关于期末考试安排的通知",
  content: "详细内容...",
  author: "教务处",
  time: "2024-01-01",
  isRead: false
}
```

4.2 创建小程序项目

步骤 1：新建项目

1. 打开微信开发者工具
2. 新建小程序项目
3. 填写 AppID（如无则使用测试号）
4. 选择 JavaScript 模板

4.3 AI 辅助编码

步骤 1：生成列表页代码

1. 使用 TRAE 描述需求：微信小程序通知列表页面，WXML 列表渲染，点击跳转详情页
2. AI 生成代码片段
3. 手动完善

步骤 2：实现通知列表页面

```
<!-- pages/index/index.wxml -->
<view class="container">
  <view class="news-list">
    <view class="news-item {{item.isRead ? "read" : ""}} wx:for="{{newsList}}"
wx:key="id" bindtap="goToDetail" data-id="{{item.id}}">
      <view class="title">{{item.title}}</view>
      <view class="info">
        <text>{{item.author}}</text>
        <text>{{item.time}}</text>
      </view>
    </view>
  </view>
</view>
</view>
/* pages/index/index.wxss */
.news-item {
```

```

padding: 16px;
border-bottom: 1px solid #eee;
background: #fff;
}
.news-item.read {
  opacity: 0.6;
}
.title {
  font-size: 16px;
  font-weight: bold;
  margin-bottom: 8px;
}
.info {
  font-size: 12px;
  color: #999;
  display: flex;
  justify-content: space-between;
}
// pages/index/index.js
Page({
  data: { newsList: [] },
  onLoad() { this.loadNews(); },
  onPullDownRefresh() {
    this.loadNews().then(() => { wx.stopPullDownRefresh(); });
  },
  loadNews() {
    const newsList = [
      { id: 1, title: "关于期末考试安排的通知", content: "详细内容...", author: "教务处", time:
"2024-01-01", isRead: false },
      { id: 2, title: "图书馆开放时间调整", content: "详细内容...", author: "图书馆", time:
"2024-01-02", isRead: false },
      { id: 3, title: "校园网络维护通知", content: "详细内容...", author: "信息中心", time:
"2024-01-03", isRead: false }
    ];
    const readIds = wx.getStorageSync("readIds") || [];
    newsList.forEach(news => { news.isRead = readIds.includes(news.id); });
    this.setData({ newsList });
    return Promise.resolve();
  },
  gotoDetail(e) {
    const id = e.currentTarget.dataset.id;
    wx.navigateTo({ url: `/pages/detail/detail?id=${id}` });
  }
});
// pages/index/index.json
{

```

```
"navigationBarTitleText": "智慧校园通知",
"enablePullDownRefresh": true
}
```

项目结构（符合架构图）

```
pages/
├── index/
│   ├── index.wxml # View Layer: 列表页面结构
│   ├── index.wxss # View Layer: 列表页面样式
│   ├── index.js   # Logic Layer: 列表页面逻辑
│   └── index.json # Config Layer: 列表页面配置
└── detail/
    ├── detail.wxml # View Layer: 详情页面结构
    ├── detail.wxss # View Layer: 详情页面样式
    ├── detail.js   # Logic Layer: 详情页面逻辑
    └── detail.json # Config Layer: 详情页面配置
app.js      # 应用入口逻辑
app.json    # 应用全局配置
app.wxss    # 应用全局样式
```

第二课时：详情页、存储与发布（2 学时）

4.5 详情页开发

步骤 1：AI 辅助生成详情页

1. 使用 TRAE 描述需求：微信小程序通知详情页，接收参数显示内容
2. 生成代码片段

步骤 2：实现详情页

```
<!-- pages/detail/detail.wxml -->
<view class="container">
  <view class="title">{{news.title}}</view>
  <view class="info">
    <text>发布部门：{{news.author}}</text>
    <text>发布时间：{{news.time}}</text>
  </view>
  <view class="content">{{news.content}}</view>
</view>
// pages/detail/detail.js
Page({
  data: { news: {} },
  onLoad(options) {
    const id = parseInt(options.id, 10);
    this.loadDetail(id);
  },
});
```

```

loadDetail(id) {
  const allNews = [
    { id: 1, title: "关于期末考试安排的通知", content: "本次期末考试安排如下...", author: "教务处", time: "2024-01-01" },
    { id: 2, title: "图书馆开放时间调整", content: "图书馆开放时间调整...", author: "图书馆", time: "2024-01-02" },
    { id: 3, title: "校园网络维护通知", content: "网络维护通知...", author: "信息中心", time: "2024-01-03" }
  ];
  const news = allNews.find((n) => n.id === id) || {};
  this.setData({ news });
  this.markAsRead(id);
},
markAsRead(id) {
  const readIds = wx.getStorageSync("readIds") || [];
  if (!readIds.includes(id)) {
    readIds.push(id);
    wx.setStorageSync("readIds", readIds);
  }
}
});

```

4.6 本地存储

步骤 1：实现已读状态存储

// 保存已读 ID

```
wx.setStorageSync("readIds", [1, 2, 3]);
```

// 读取已读 ID

```
const readIds = wx.getStorageSync("readIds");
```

// 检查是否已读

```
const isRead = readIds.includes(newsId);
```

4.7 测试验证

步骤 1：真机预览

1. 编译项目
2. 扫描二维码预览
3. 测试各项功能

步骤 2：功能测试

测试用例 1：通知列表展示测试

测试项	测试内容
-----	------

测试场景	小程序启动后正常加载通知列表数据
测试步骤	1. 启动小程序 2. 等待数据加载 3. 查看列表展示
预期结果	成功加载并展示通知列表
实际结果	☒ 符合预期，列表正常展示

测试用例 2：详情页跳转测试

测试项	测试内容
测试场景	点击列表项跳转到详情页
测试步骤	1. 点击任意通知列表项 2. 查看详情页内容
预期结果	成功跳转到详情页，显示对应通知内容
实际结果	☒ 符合预期，跳转正常

测试用例 3：已读状态存储测试

测试项	测试内容
测试场景	查看通知后已读状态持久化存储
测试步骤	1. 点击通知进入详情页 2. 返回列表页 3. 重启小程序验证已读状态
预期结果	已读通知显示半透明效果，重启后状态保持
实际结果	☒ 符合预期，已读状态正常保存

测试真机图片



图 4. 小程序真机测试截图

4.8 部署运维

步骤 1：体验版发布

1. 点击上传
2. 填写版本信息
3. 获取体验版二维码

步骤 2：审核发布流程了解

1. 了解审核要求
2. 准备发布材料
3. 体验完整流程

总结与思考

一、实验总结

本次实验通过实现"智慧校园通知"小程序，深入掌握了微信小程序开发的核心技术：

1. 小程序三层架构设计实践

实现了 **View Layer**（视图层）、**Logic Layer**（逻辑层）、**Config Layer**（配置层）的清晰分层

小程序采用 **Page()** 注册页面，**WXML/WXSS** 负责 UI，**JS** 负责逻辑，**JSON** 负责配置
代码结构符合小程序开发规范，便于维护和扩展

2. WXML/WXSS/JS 核心技术掌握

掌握了 **wx:for** 列表渲染、数据绑定 **{{}}**、事件绑定 **bindtap** 等核心语法

实现了页面路由 **wx.navigateTo** 与参数传递

完善了下拉刷新 **onPullDownRefresh** 等生命周期函数的使用

3. 本地存储数据持久化

掌握了 **wx.setStorageSync/wx.getStorageSync** 的使用

实现了已读状态的持久化存储

理解了小程序本地存储的限制与最佳实践

4. AI 辅助开发体验

使用 **TRAE** 辅助生成页面代码，提升开发效率

学会了如何描述需求让 **AI** 生成符合规范的代码

理解了 **AI** 辅助与人工调试相结合的开发模式

二、实现难点与解决方案

1. 页面路由与参数传递

难点：如何在页面间传递数据并正确接收

解决：使用 **wx.navigateTo** 传递 **URL** 参数，在 **onLoad** 中通过 **options** 接收

2. 已读状态的持久化

难点：如何在小程序重启后保持已读状态

解决：使用 **wx.setStorageSync** 存储已读 **ID** 数组，加载时读取并标记

3. 列表渲染与性能优化

难点：大量数据时的列表性能问题

解决：使用 **wx:key** 提升渲染性能，合理设置 **item** 样式

三、性能优化策略

1. 列表性能优化

使用 **wx:key** 避免不必要的节点重建

合理使用 **setData**，避免频繁更新

图片懒加载（实际项目中）

2. 存储优化

合理设计存储数据结构

注意存储大小限制（单个 **key** 最大 **1MB**）

定期清理不必要的存储数据

3. 启动速度优化

分包加载（大型项目）

延迟加载非首屏数据

使用骨架屏提升加载体验

四、未来功能扩展建议

1. 功能增强

通知搜索与筛选功能
通知分类与标签管理
推送通知（新通知提醒）
通知收藏功能

2. 技术升级

接入后端 **API**，实现真实数据请求
实现数据分页加载（上拉加载更多）
集成云开发（**Cloud Base**）
添加用户登录与权限管理

3. 架构演进

引入状态管理（**MobX**、**Redux** 等）
实现组件化开发，提升代码复用
添加单元测试
CI/CD 自动化构建与部署

五、课后思考

1. 小程序相比原生 **App** 的优势

无需安装，即用即走的用户体验
开发成本低，跨平台兼容性好
微信生态深度整合，社交传播便利

2. 如何利用 **AI** 工具提升开发效率

AI 代码生成与人工调试相结合
学会清晰描述需求，让 **AI** 生成更准确的代码
AI 辅助文档生成与测试用例设计

3. 小程序在校园场景的创新应用

校园一卡通、课表查询、成绩查询
校园活动报名、二手交易、失物招领
校园社交、学习互助、就业信息